

PROJECT-Q Workbook – 08

30-Day Quantum Computing Challenge

DAY – 08

Understanding Qiskit Architecture

Learning Objectives

By the end of this chapter, you will be able to:

- ✓ Understand what Qiskit is and why it is important.
 - ✓ Learn the architecture of the Qiskit ecosystem.
 - ✓ Understand how a quantum program travels from Python code to a quantum computer.
 - ✓ Differentiate between simulators and real quantum hardware.
 - ✓ Understand the role of IBM Quantum Platform in executing quantum programs.
-

Quick Recap

Before today's lesson, you have learned:

- A **qubit** stores quantum information.
- Quantum gates modify qubit states.
- Measurement converts quantum information into classical information.
- Quantum circuits are built by applying gates to qubits.
- In the previous lab, you built a Quantum Random Number Generator (QRNG) using a simple quantum circuit.

Today's focus is not on quantum mechanics—it's on the **software ecosystem** that allows us to program quantum computers.

Why Do We Need Qiskit?

Imagine you want to build a mobile application.

You don't directly communicate with the processor using binary instructions.

Instead, you use tools like:

- Programming languages
- Libraries
- Compilers
- Debuggers

These tools simplify software development. Quantum computing works in the same way.

Instead of writing low-level instructions for a quantum processor, we use **Qiskit**, a software development kit (SDK) that provides everything needed to design, optimize, execute, and analyze quantum programs.

Without Qiskit, programming quantum computers would be extremely difficult.

What is Qiskit?

Qiskit is IBM's open-source **Software Development Kit (SDK)** for quantum computing.

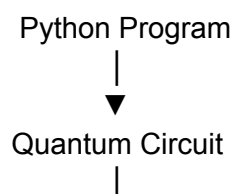
It allows developers, researchers, and students to:

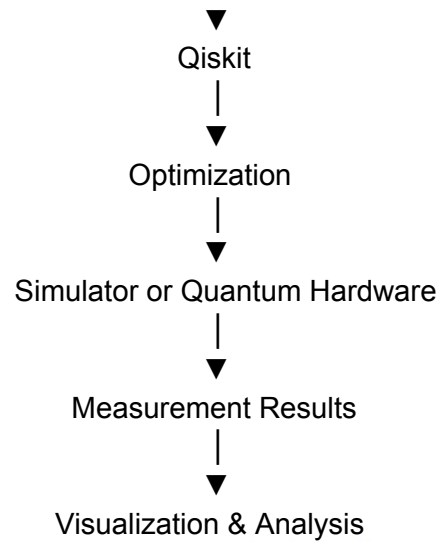
- Design quantum circuits
- Simulate quantum algorithms
- Execute programs on real quantum computers
- Analyze and visualize quantum results

Think of Qiskit as the bridge between **Python** and a **quantum computer**.

Understanding the Big Picture

Every quantum program follows the same journey.

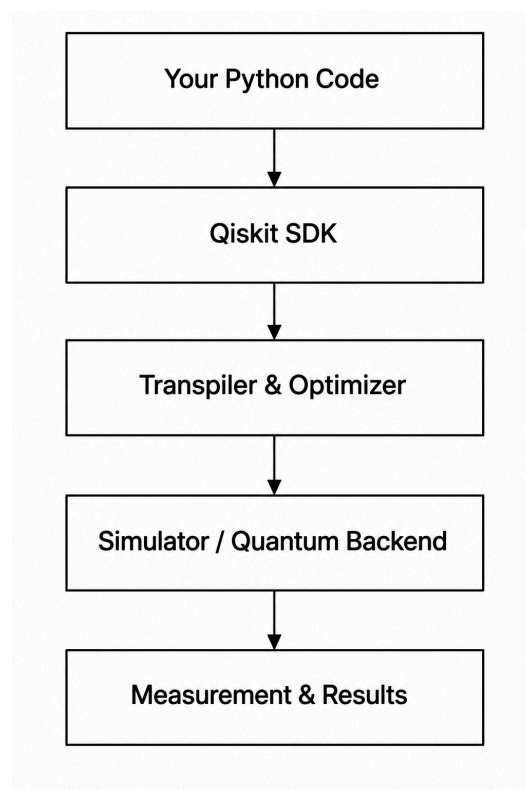




Although this workflow looks simple, many software components work together behind the scenes.

The Qiskit Architecture

Let's break the architecture into layers.



Each layer has a specific responsibility. Let's understand them one by one.

Layer 1 - Your Python Program

Everything begins with Python.

Example:

```
from qiskit import QuantumCircuit
qc = QuantumCircuit(1,1)
qc.h(0)
qc.measure(0,0)
```

This is the code you write. It describes **what** you want the quantum computer to do. It does **not** directly communicate with the hardware.

Layer 2 - Qiskit SDK

The SDK acts as the central software layer.

Its responsibilities include:

- Creating circuits
- Managing qubits
- Providing quantum gates
- Connecting to simulators
- Connecting to IBM Quantum hardware
- Visualizing results

You can think of the SDK as a toolbox containing everything required to build quantum applications.

Layer 3 - The Transpiler

Different quantum computers have different hardware layouts. Some gates may not be directly supported. The **transpiler** automatically prepares your circuit for the selected backend.

It performs tasks such as:

- Optimizing the circuit
- Reducing unnecessary gates
- Adapting the circuit to the hardware's connectivity
- Improving execution efficiency

Why is this important?

Imagine writing software for Windows and then running it on a Mac. Some adjustments are necessary. The transpiler performs a similar role for quantum hardware.

Layer 4 - Backend

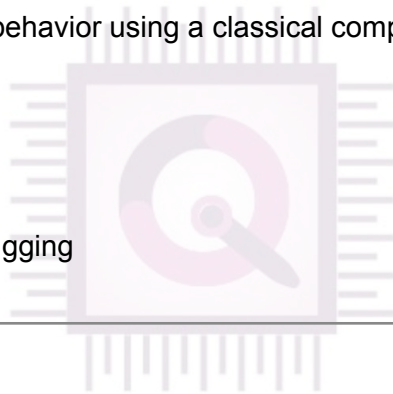
A **backend** is the system that executes your quantum circuit. Qiskit allows two types of backends.

Simulator

A simulator reproduces quantum behavior using a classical computer.

Advantages:

- Fast
- Free
- Ideal for learning and debugging



Real Quantum Hardware

Real quantum processors execute your circuit using physical qubits.

Advantages:

- True quantum computation
- Real hardware experience

Challenges:

- Noise
- Decoherence
- Queue times
- Limited qubit availability

As beginners, you will mostly work with simulators before moving to real hardware.

Layer 5 - Results

Once execution is complete, qiskit returns measurement data.

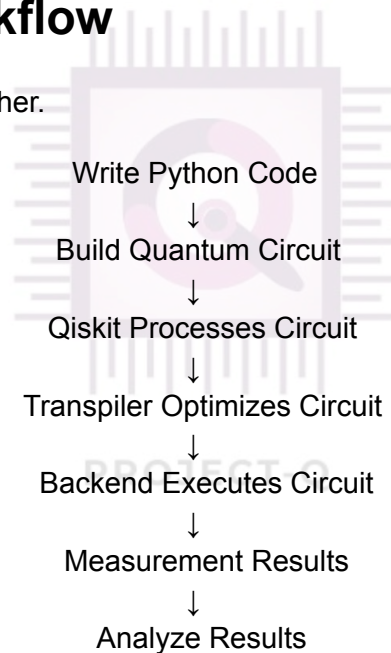
Example:

```
{'0': 503,  
'1': 497}
```

These results can then be visualized using charts and histograms. This is the final output of the quantum workflow.

The Complete Workflow

Let's summarize everything together.



This is the workflow followed by almost every quantum application.

Simulators vs Real Quantum Computers

Simulator	Real Quantum Computer
Runs on classical hardware	Uses physical qubits
Fast execution	Slower due to queue times
No hardware noise (ideal simulation)	Affected by quantum noise
Best for learning	Best for real experiments

Both are essential. Students generally develop and test programs using simulators before running them on actual quantum processors.

IBM Quantum Platform

The IBM Quantum Platform provides access to:

- Quantum simulators
- Real quantum computers
- Learning resources
- Circuit Composer
- Job management
- Result visualization

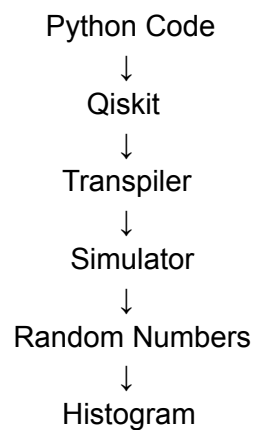
Instead of installing everything locally,

you can access quantum hardware directly through your IBM Quantum account.

This makes quantum computing accessible to anyone with an internet connection.

Where Does the QRNG Fit?

Let's revisit the project from Day 07.



Notice that every component of the architecture participated in generating those random numbers. You were already using the Qiskit architecture you just didn't know what was happening behind the scenes.

Common Beginner Mistakes

"Qiskit is a quantum computer."

✓ No.

Qiskit is a software development kit used to program quantum computers.

"Python directly controls quantum hardware."

✓ No.

Python code is converted into quantum instructions before execution.

"A simulator is the same as a quantum computer."

✓ Simulators imitate quantum behavior, while real quantum computers execute operations on physical qubits and are affected by hardware limitations.

"The transpiler changes the algorithm."

✓ The transpiler preserves the algorithm's logic while rewriting the circuit into a form that the selected hardware can execute efficiently.

Remember This

- Qiskit is IBM's open-source SDK for quantum computing.
 - Every quantum program follows a software workflow from Python to hardware.
 - The transpiler prepares circuits for specific quantum devices.
 - A backend is where the circuit is executed.
 - Results are returned as measurement counts and can be visualized using Qiskit tools.
-

End-of-Day Summary

Today, you learned:

- What Qiskit is.
- Why a software development kit is needed.

- The overall Qiskit architecture.
 - The role of the transpiler.
 - What a backend is.
 - The difference between simulators and real quantum hardware.
 - How a quantum program travels from Python code to execution.
-

Top Verified Learning Resources

1. IBM Quantum Documentation (Official)

Introduction to IBM Quantum Tools

<https://quantum.cloud.ibm.com/docs/en/guides/tools-intro>

Recommended sections:

- IBM Quantum Platform
- Qiskit SDK
- Backends
- Execution workflow
- Simulators and hardware



2. Video Resource

Understanding Qiskit Architecture

<https://www.youtube.com/watch?v=fMH2Zn6uO4s>

This video provides a beginner-friendly overview of the Qiskit ecosystem, explaining how Python code is transformed into executable quantum circuits.

Qiskit Architecture Explained

<https://www.youtube.com/watch?v=NTpIT4WnNbk>

A visual walkthrough of the Qiskit workflow, including the role of the transpiler, backends, and execution pipeline.